

Implementation of First Come First Serve Algorithm for Elective Registration

Soni Priyansee Mukeshkumar
MTech (ICT)

Dhirubhai Ambani University
Gandhinagar, Gujarat, India
202511011@dau.ac.in

Prof. P M Jat
Guide

Dhirubhai Ambani University
Gandhinagar, Gujarat, India
pm_jat@daiict.ac.in

Abstract—Elective registration in universities appears deceptively routine until the moment the portal opens and several hundred students attempt to claim limited seats within the same few seconds. This report documents the design and deployment of a Subject Allocation System built for Term 78 at Dhirubhai Ambani University. The system adopts a transaction-driven First-Come-First-Served (FCFS) strategy, anchored by PostgreSQL row-level pessimistic locking. Two recurring failure modes in concurrent registration software the *lost update anomaly* and *thread barging* shaped every major design decision. The central insight is that wrapping the seat-check and seat-decrement within a single `SELECT FOR UPDATE` transaction collapses an otherwise hazardous read-modify-write window into a single serializable unit, with no need for application-layer mutexes or version counters. A JUnit stress test pitting 100 simultaneous threads against a 20-seat slot confirmed the guarantee empirically: exactly 20 allocations succeeded, zero overbookings were recorded, and no duplicate enrollment records appeared in the database.

Index Terms—Spring Boot, FCFS Algorithm, Pessimistic Locking, Two-Phase Locking, PostgreSQL, Serializability, Concurrency Control, Elective Registration

I. INTRODUCTION

Among the many kinds of shared-state operations that academic information systems must handle, elective registration stands apart for one structural reason: it is deliberately concentrated in time. Administrators open a window, students rush in, and within minutes sometimes seconds most seats are gone. Unlike routine administrative writes distributed across a working day, registration is an engineered spike, and the database layer has no choice but to absorb it without error.

The technical difficulty stems from a gap between reading a value and acting on it. Checking seat availability and then decrementing the count are two database operations, not one. Without explicit coordination, two transactions can read the same non-zero count, independently conclude that a seat exists, and proceed to insert enrollment records simultaneously leaving the course over-enrolled. This is what the concurrency literature calls the *lost update* problem, and it does not disappear just because the application logic looks correct in isolation.

Fairness adds a second dimension. A student who clicks “Register” at 09:00:01 should, by any reasonable definition of a first-come policy, receive a seat before one who clicks

at 09:00:03. Systems that rely on optimistic locking with unconstrained retries can invert this ordering: a request that loses a conflict on the first attempt may win on its retry simply by getting rescheduled ahead of the original winner. This overtaking behaviour borrowing the term *barging* from lock-based concurrency theory is not a theoretical edge case but a real consequence of retry-without-ordering.

The design described in this report addresses both problems through a single mechanism: holding all necessary row-level locks before performing any write. This forces serialization at the database layer and, as a natural side effect, processes requests in the order locks are granted which tracks arrival order closely enough for practical purposes.

The report is organized as follows. Section II states the problem formally. Section III surveys the relevant theoretical and practical literature. Section IV describes the system architecture, Section V the database schema. Section VI details the three-phase allocation algorithm, and Sections VII and VIII cover concurrency handling and implementation specifics. Section IX reports experimental results, including a subtle caching bug uncovered during development. Sections X and XI discuss limitations and future directions, and Section XII concludes.

II. PROBLEM STATEMENT

A. Lost Update Problem

Consider a course with exactly one seat remaining. Students S_1 and S_2 both submit registration requests within milliseconds of each other. Both transactions read `seats_available = 1` before either has written anything. Both consider the seat unclaimed. Both then decrement the counter and insert enrollment records. The table ends up reflecting two valid registrations against one physical seat.

Formalizing the hazardous interleaving:

$$R_1(s) \rightarrow R_2(s) \rightarrow W_1(s) \rightarrow W_2(s) \quad (1)$$

where s is the seat counter. A safe schedule requires the operations of T_1 and T_2 to be non-overlapping: one transaction must complete its read-write pair before the other begins. Any interleaving that violates this produces a state invisible to either transaction individually but erroneous from the database’s perspective.

B. Barging Problem

Preventing overbooking through optimistic locking introduces a secondary hazard. When two transactions conflict and one is forced to retry, the retry competes afresh with no memory of the original submission time. Whichever thread wins the CPU next claims the seat, regardless of which student clicked first. In an academic context where registration priority can translate directly into graduation timelines this is not a minor quality-of-service issue. It is a policy violation embedded in the system’s concurrency design.

C. Concurrency Anomalies Addressed

Table I lists the four anomalies considered during design and the chosen mitigation for each.

TABLE I
CONCURRENCY ANOMALIES AND MITIGATIONS

Anomaly	Description	Mitigation
Lost Update	Concurrent writes overwrite each other	Pessimistic row lock
Dirty Read	Reading uncommitted changes	@Transactional isolation
Barging	Out-of-order seat allocation	FCFS lock ordering
Deadlock	Circular lock wait	Sorted ID acquisition

III. LITERATURE SURVEY

A. Two-Phase Locking (2PL)

The formal underpinning of safe concurrent transactions goes back to Eswaran et al. (1976) [7], whose Two-Phase Locking (2PL) theorem partitions a transaction’s lifetime into a *growing phase* in which only lock acquisitions occur and a *shrinking phase* in which locks are released but never re-acquired. The result is that any schedule produced by 2PL-disciplined transactions is conflict-serializable. More recently, Martinenghi (2025) proposed a visual analysis framework that simplifies reasoning about 2PL schedule membership without fully expanding the serialization graph, an approach useful for teaching and audit purposes [8].

B. Pessimistic vs. Optimistic Locking

The academic debate between pessimistic and optimistic schemes is well-worn, but its resolution depends heavily on workload shape. Optimistic locking performs well when conflicts are rare a common observation in read-heavy environments. It degrades, sometimes sharply, under sustained write contention. Chiu et al. (1998) showed that in real-time multiprocessor databases, optimistic schemes under high contention spend a disproportionate share of execution time on conflict detection and re-execution overhead rather than useful work [1]. Roztropiński (2025) revisited this tradeoff in a modern Spring/JPA context and concluded that optimistic locking’s retry cost can dominate response time whenever many writers converge on the same rows concurrently [3]. A registration window is precisely this scenario: a concentrated, write-heavy burst on a small set of seat-counter rows.

C. FCFS as a Formal Property

Hesselink, Mathijssen, and Vervoort (2024) offered a clean operational characterization of the FCFS property as a two-part progress guarantee [9]. A *Doorway* step which must complete in finite time with no waiting is followed by a *Waiting* step. The FCFS condition requires that any process completing its Doorway before another cannot be overtaken by that other in acquiring the resource. This maps onto the three-phase design in Section VI: Phase 0 functions as the Doorway (entirely wait-free), and Phases 1–2 together constitute the Waiting and critical-section execution. Barging is ruled out because lock acquisition ordering mirrors Doorway completion ordering under PostgreSQL’s queuing policy.

D. PostgreSQL Explicit Locking

The database mechanism central to this work is `SELECT FOR UPDATE` [4]. When issued inside an open transaction, it places an exclusive row-level lock that no other transaction can challenge until the holder commits or rolls back. PostgreSQL also maintains a lock-wait graph and runs a background deadlock detector that forcibly aborts one participant in any detected wait cycle [5]. The sorted-ID acquisition strategy described in Section VII makes actual deadlock occurrence impossible, but the detector remains a safety net against implementation errors.

IV. SYSTEM ARCHITECTURE

The application is structured in four layers, each with a distinct responsibility and no dependency inversion across adjacent tiers.

- **Presentation Layer:** Thymeleaf templates render the slot-selection form and the post-allocation confirmation page. No business logic leaks into the templates; they bind to model attributes populated by the controller.
- **Controller Layer:** `RegistrationController` handles browser-facing model-and-view flows, while `SlotController` exposes REST endpoints for programmatic access. Both are thin: they validate HTTP-level inputs, delegate to the service layer, and map results to responses.
- **Service Layer:** `AllocationService` owns the @Transactional boundary and contains the three-phase FCFS logic. `SlotService` handles read-only slot queries in a separate service to avoid coupling seat-inventory reads to the locking transaction.
- **Persistence Layer:** Spring Data JPA repositories backed by Hibernate [2] and PostgreSQL persist all state to the `ec2` schema. HikariCP manages the connection pool, which is capped at 20 connections for the test environment.

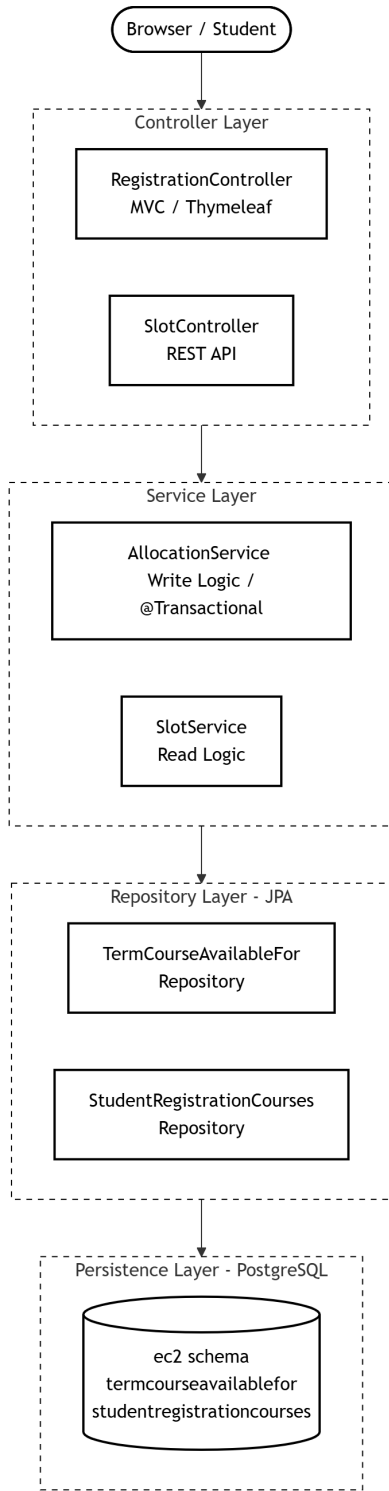


Fig. 1. Layered architecture of the Subject Allocation System.

One configuration decision worth noting explicitly: `ddl-auto=validate`. Rather than permitting Hibernate to silently alter or recreate tables on startup, this setting causes the application to refuse to boot if any entity definition diverges from the live schema. In a shared academic database where other applications depend on the same tables,

a stray `ddl-auto=update` could corrupt production data irreversibly.

V. DATABASE DESIGN

A. Schema Overview

Five tables in the `ec2` schema carry all registration state, each mapped to a Java entity through Hibernate [2].

- **courses**: The master subject catalog, holding course codes, names, credit weights, and departmental assignments. Treated as read-only during active registration windows.
- **termcourses**: Associates a course with a specific term and time slot. A given subject can appear in multiple terms without duplication in the `courses` table.
- **termcourseavailablefor (TCAF)**: Carries seat inventory per student-group-course combination via two columns: `tca_seats` (the configured capacity) and `tca_booked` (confirmed enrollments so far). TCAF rows are the rows locked during the allocation transaction.
- **studentregistrationcourses (SRC)**: Each row represents one confirmed enrollment. A row only ever appears here after the enclosing transaction has held its locks, confirmed availability, and committed so the table is free of phantom or rolled-back records by construction.

B. Key Design Decision

Seat inventory was placed in TCAF rather than in `termcourses`. This was intentional. Lock granularity therefore falls at the student-group-course level rather than the course level. Two students competing for courses in different student groups even if those courses share the same `termcourses` row will never block each other. Under the alternative schema, every registration attempt for a popular subject would serialize all groups, inflating average wait time well beyond what the algorithm itself requires.

VI. METHODOLOGY / ALLOCATION ALGORITHM

A. Algorithm Overview

The allocation runs in three sequential phases. The governing invariant is that no irreversible write neither a `tca_booked` increment nor an SRC insert may occur until every necessary lock is already held. Violating this ordering, even for a single row, re-opens the lost-update window the algorithm is meant to close.

B. Phase 0: Validation (Wait-Free)

Instead of loading full TCAF entities, this phase calls `findTcaIdsBySlot(slotId)`, which projects only primary keys into a `List<Long>`. The distinction is critical: loading complete entities at this stage would populate Hibernate's first-level (session) cache, and Phase 1's subsequent `SELECT FOR UPDATE` calls would then be answered from that local cache rather than from PostgreSQL, defeating the lock entirely. The phase also checks that the student holds no existing allocation in this slot. Because Phase 0 acquires

no locks, it satisfies the wait-free Doorway requirement from Hesselink et al. [9].

C. Phase 1: Growing Phase (Lock Acquisition)

Before acquiring any lock, the ID list is sorted in ascending order using `Collections.sort()`. Every concurrent thread follows the same global ordering, which eliminates circular-wait by construction. Locks are then taken one at a time via `findTcafByIdWithLock(tcaId)`, which issues `SELECT ... FOR UPDATE` to PostgreSQL. The phase does not exit until the final lock in the sorted list has been granted. This is the Waiting step: a thread that arrived later and therefore has a later lock-acquisition timestamp cannot overtake a thread already holding earlier-acquired locks in the sequence.

D. Phase 2: Modify Phase (Atomic Write)

With all locks in hand, `tca_booked` is incremented and a matching SRC entity is saved for each course in the slot. The `@Transactional` boundary [6] ensures these writes form a single atomic unit: if the insert for the third course of a five-course slot fails for any reason, the increments already applied to courses one and two are rolled back along with everything else. Partial registrations are therefore structurally impossible.

Algorithm 1 presents the complete pseudocode.

Algorithm 1 FCFS Seat Allocation

Require: studentId, slotId

Ensure: Atomic and fair seat allocation

```

1: // Phase 0: Validation (Wait-Free)
2: Fetch TCAF ID list for slotId (primary keys only)
3: if student already enrolled in slot then
4:   return ALREADYENROLLEDException
5: end if
6: Sort TCAF IDs ascending
7: // Phase 1: Lock Acquisition (Growing Phase)
8: for each tcaId in sorted ID list do
9:   SELECT FOR UPDATE on tcaId
10: end for
11: // Phase 2: Atomic Modify
12: if tca_booked < tca_seats then
13:   Increment tca_booked
14:   Insert into studentregistrationcourses
15:   Commit transaction
16: return ALLOCATION SUCCESSFUL
17: else
18:   Rollback transaction
19:   return RUNTIMEException("SLOT FULL")
20: end if

```

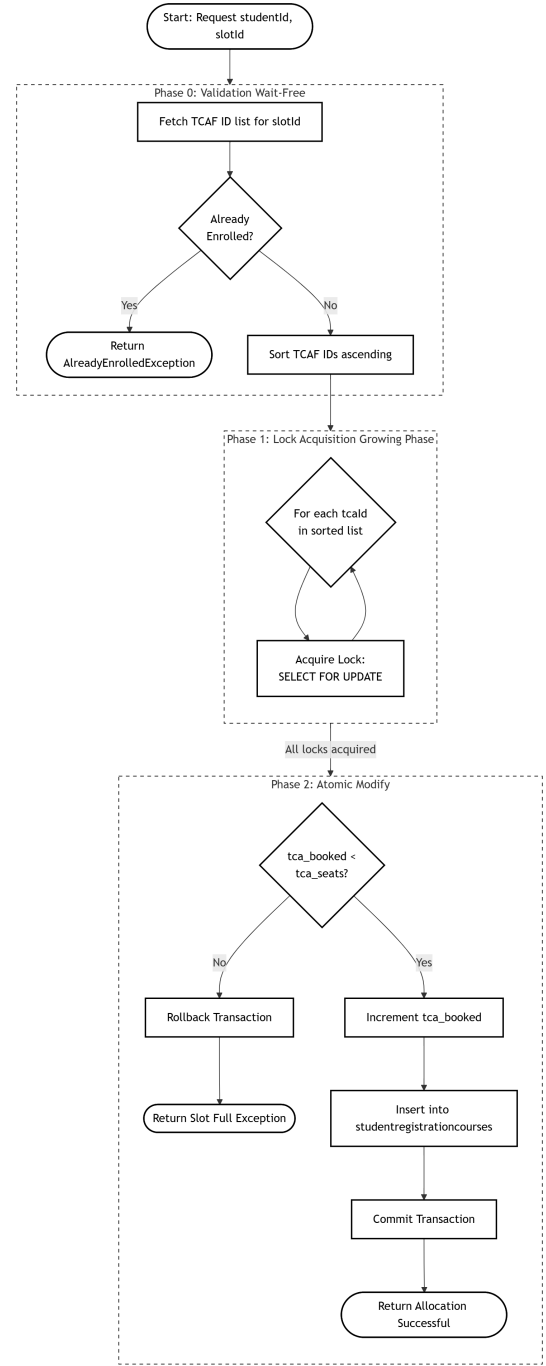


Fig. 2. Three-phase FCFS allocation algorithm flowchart.

VII. CONCURRENCY HANDLING

A. Pessimistic Write Locking

Once a transaction issues `SELECT FOR UPDATE` on a TCAF row, PostgreSQL places an exclusive lock that any competing transaction must wait behind [4]. This has an important practical consequence: the three logically separate steps read seat count, check availability, write increment are experienced by any observer outside the lock holder as a single indivisible operation. The gap that normally exists between

the read and the write, the gap that permits the lost-update interleaving in Equation (1), is sealed.

B. Deadlock Prevention via Ordered Lock Acquisition

A deadlock in this system would require Thread A to hold the lock on row 341 while waiting for row 342, and Thread B to hold row 342 while waiting for row 341. Sorting TCAF IDs before the first acquisition makes this cycle structurally unreachable:

```
1 List<Long> tcaIdsToLock =
2   termCourseAvailableForRepository
3   .findTcaIdsBySlot(slotId);
4 Collections.sort(tcaIdsToLock); // Deadlock prevention
```

Listing 1. Sorted lock acquisition in AllocationService

When every thread unconditionally acquires locks in ascending ID order say, 341 then 342 then 343 no two threads can be waiting on each other’s held lock. PostgreSQL’s deadlock detector therefore never fires during normal operation, though it remains available as a last-resort safety net [5].

C. Atomicity via Spring Transactions

The `@Transactional` annotation on `allocateSlot` [6] creates a single commit boundary around all writes in a call. For a slot containing, say, five courses, all five `tca_booked` increments and five SRC inserts commit or roll back together. A failure on the fourth iteration does not leave the student partially registered in three courses it leaves them registered in none, which is the only logically coherent outcome.

VIII. IMPLEMENTATION DETAILS

A. Technology Stack

Table II summarizes the tools and versions deployed.

TABLE II
TECHNOLOGY STACK

Component	Technology
Language	Java 21
Framework	Spring Boot 3.5
ORM	Hibernate 6 / Spring Data JPA
Database	PostgreSQL (ec2 schema)
Connection Pool	HikariCP (max 20 connections)
Template Engine	Thymeleaf
Build Tool	Maven

B. Schema Safety

Setting `spring.jpa.hibernate.ddl-auto=validate` was an early call that paid dividends during iterative development. On two occasions during schema evolution, a mismatched column name caused the application to refuse startup rather than silently proceeding with an incompatible mapping. In a shared academic database environment, the alternative silent migration could have altered tables depended upon by other services.

C. REST API Endpoints

`SlotController` exposes two endpoints:

- GET `/api/slots` returns available slots and per-course seat counts filtered to the requesting student’s eligibility group.
- POST `/api/register` accepts a `{studentId, slotId}` payload, delegates to `AllocationService`, and returns HTTP 200 on success, HTTP 409 when the requested slot is full, and HTTP 400 for input validation failures.

All queries use Spring Data JPA `@Param` bindings or parameterized `JdbcTemplate` calls. Concatenated SQL strings were never used, so SQL injection is not an attack surface.

IX. EXPERIMENTAL RESULTS

A. Functional Testing

Each allocation scenario was first exercised in isolation before any concurrent load was applied. Table III records the outcomes.

TABLE III
FUNCTIONAL TEST CASES

Test Case	Condition	Expected Output	Status
Valid Allocation	Seats > 0, valid IDs	Allocation Successful	PASS
Invalid Slot	Slot does not exist	SlotNotFoundException	PASS
Student Not Found	Unregistered student ID	RuntimeException	PASS
Already Enrolled	Existing records in SRC	AlreadyEnrolledException	PASS
Slot Fully Booked	<code>tca_booked</code> ≥ <code>tca_seats</code>	RuntimeException (Slot Full)	PASS

B. Concurrency Test Setup

The stress test, `ConcurrencyTest.java`, uses an `ExecutorService` backed by a fixed thread pool paired with a `CountDownLatch` initialized to one. Each of the 100 worker threads calls `latch.await()` before doing anything else, blocking the entire pool at a common starting gate. A single `latch.countDown()` from the main thread releases all 100 threads simultaneously, producing the maximum contention achievable on the JVM without introducing artificial delays. The slot under test was configured with 20 seats distributed across 5 subjects.

C. Concurrency Test Results

Table IV records the measured outcomes.

TABLE IV
CONCURRENCY TEST EXECUTION RESULTS

Metric	Value
Total Threads Fired	100
Available Seats (MAX)	20
Successful Allocations	20
Failed Allocations	80
Overbooking Observed	None

```

1 @Test
2 public void testConcurrentAllocation()
3     throws InterruptedException {
4
5     int threadCount = 100;
6     ExecutorService executor =
7         Executors.newFixedThreadPool(threadCount);
8
9     CountDownLatch latch =
10         new CountDownLatch(1);
11
12     AtomicInteger success =
13         new AtomicInteger(0);
14
15     AtomicInteger failure =
16         new AtomicInteger(0);
17
18     for (int i = 0; i < threadCount; i++) {
19
20         Long studentId = 20000L + i;
21
22         executor.execute(() -> {
23             try {
24                 latch.await();
25
26                 String result =
27                     allocationService.allocateSlot(
28                         studentId, "1");
29
30                 if (result.contains("Successful"))
31                     success.incrementAndGet();
32
33             } catch (Exception e) {
34                 failure.incrementAndGet();
35             }
36         });
37
38         latch.countDown();
39     }
40 }

```

Listing 2. Core concurrency test structure

Exactly 20 threads secured seats. The remaining 80, each blocked at a PostgreSQL row lock, eventually acquired the lock only to find `tca_booked` already equal to `tca_seats` and exited with a `RuntimeException("Slot Full")`. At no point did the bookings counter breach its configured ceiling.

D. Database Consistency Verification

Two post-run queries were issued directly against PostgreSQL to confirm the database state matched the application-level counters.

```

1 SELECT tcaid, tca_seats, tca_booked
2 FROM ec2.termcourseavailablefor
3 WHERE tca_booked > tca_seats;
4 -- Expected: 0 rows

```

Listing 3. Overbooking check query

```

1 SELECT srscrgid, COUNT(DISTINCT srctcrd)
2 FROM ec2.studentregistrationcourses
3 GROUP BY srscrgid
4 HAVING COUNT(DISTINCT srctcrd) > 5;
5 -- Expected: 0 rows

```

Listing 4. Duplicate registration check

Both returned zero rows. No student was registered in more courses than the slot contained, and no course held more bookings than declared.

E. Critical Bug: Hibernate Session Cache Bypassing Row Locks

One of the more instructive problems encountered during development involved Hibernate’s first-level cache silently defeating the pessimistic lock.

Root Cause. The original Phase 0 called `findTcafBySlot(slotId)`, loading complete TCAF entities. Hibernate stores these in the session-level identity map. When Phase 1 subsequently issued `findTcafByIdWithLock(tcaId)` for the same primary keys, Hibernate returned the cached instances without contacting PostgreSQL at all. The `SELECT FOR UPDATE` was never executed; the application compared against a `tca_booked` value that could already be stale by the time any evaluation happened.

Fix. Phase 0 was refactored to call `findTcaIdsBySlot(slotId)`, projecting only a `List<Long>`. With no entity in the session cache, each `findTcafByIdWithLock` call in Phase 1 is a genuine database round-trip, and the `SELECT FOR UPDATE` reaches PostgreSQL as intended. Without this single-line change in the repository query, the stress test results in Table IV could not be reproduced consistently.

X. CHALLENGES & LIMITATIONS

A. Lock Acquisition Order vs. Arrival Order

The system serializes concurrent registrations by the order in which their transactions acquire the first row lock not by the wall-clock timestamp of the originating HTTP request. Between the moment a student submits the form and the moment the corresponding thread reaches `SELECT FOR UPDATE`, variable amounts of network latency, Tomcat thread-pool scheduling delay, and JVM garbage-collection pauses may have elapsed. Two requests arriving one millisecond apart can acquire locks in either order. Closing this gap fully would require timestamping at the network ingress point and enforcing explicit queue discipline before any database access, which is a meaningful architectural addition beyond what this project scope covered.

B. Connection Pool as a Secondary Bottleneck

With HikariCP capped at 20 connections, threads 21 through 100 in the stress test must wait for a database connection before they can even begin waiting for a row lock. This two-stage queuing produced intermittent `Connection is not available`, request timed out exceptions during the 100-thread run at default pool settings. Average response time also grows roughly linearly with the depth of the lock queue, since every waiting transaction must hold a connection for the full duration of its wait.

C. Single Relational Node Assumption

Pessimistic locking works because all transactions contend against a single PostgreSQL lock manager. Two application nodes pointing to the same database server present no problem. However, any form of database sharding or read-replica

redirection for writes would fragment the lock namespace and silently break mutual exclusion. Extending the design to a genuinely distributed deployment would require replacing PostgreSQL row locks with a distributed coordination mechanism.

XI. FUTURE SCOPE

A. Connection Pool Tuning

The most immediately actionable improvement is raising `spring.datasource.hikari.maximum-pool-size` to at least 50 for any deployment that expects more than 20 simultaneous active registrations. The 20-connection ceiling was adequate for controlled testing but is likely to produce visible timeout errors during an actual Term registration event at DAU, where concurrent user counts could be an order of magnitude higher.

B. Decoupling Reservation from Relational Write

`SELECT FOR UPDATE` is inherently single-threaded per contested row. That is acceptable for moderate concurrency but will degrade once the queued waiter count grows large. One viable path is to move the seat-decrement step into a Redis Lua script, which executes atomically on the Redis server without holding a relational lock. The confirmed reservation would then be persisted asynchronously through a message broker such as Kafka or RabbitMQ, trading immediate consistency for substantially higher throughput on the hot path.

C. Predicate Locking

The current implementation locks a concrete, pre-enumerated set of TCAF rows. A newly inserted TCAF row one that did not exist at the time Phase 0 ran would escape the lock set entirely. This is the phantom-read scenario that predicate locking, as formalized by Eswaran et al. [7], is designed to close by locking the search predicate rather than the matching row snapshot. Implementing this at the application layer in Spring/JPA would require explicit range locking or a move to serializable transaction isolation at the PostgreSQL level.

D. Distributed Lock Coordination

For a deployment spanning multiple database nodes, row-level locking would need to be replaced or supplemented with a distributed coordinator. Redis with the Redlock algorithm and Apache ZooKeeper with ephemeral znodes are two well-documented options. Each introduces its own failure modes split-brain under network partition for Redlock, leader-election latency for ZooKeeper that would warrant careful analysis before adoption in a production academic system.

XII. CONCLUSION

The question motivating this project was narrow and concrete: can a standard Spring Boot and PostgreSQL stack prevent overbooking and guarantee approximate FCFS fairness during a realistic elective-registration burst? Based on both design reasoning and empirical results, the answer is yes but only if the locking discipline is applied with care.

The three-phase algorithm earns its correctness from one structural rule: no write occurs until every relevant lock is held. Wrapping the seat-check and seat-decrement inside a single `SELECT FOR UPDATE` transaction collapses the read-modify-write window that permits overbooking in naive implementations. Sorted lock acquisition eliminates deadlock without runtime overhead, and `@Transactional` atomicity ensures that a mid-transaction failure cannot leave a student partially enrolled.

The Hibernate session-cache bug was the most instructive incident during development. The symptom overbooking despite correct-looking code was subtle because the locking call was being silently short-circuited by an ORM layer answering queries from its local cache. Switching Phase 0 from loading full entities to projecting only primary keys was a small change in terms of lines modified, but it was the change that made the difference between a system where overbooking was reproducible and one where 100 simultaneous threads produced exactly 20 allocations with zero anomalies. It serves as a reminder that database-level guarantees are only as strong as the application's willingness to actually reach the database.

REFERENCES

- [1] A. Chiu, B. Kao, and K. Lam, "An analysis of lock-based and optimistic concurrency control protocols in multiprocessor real-time databases," *J. Syst. Softw.*, 1998.
- [2] "Documentation 7.3 Hibernate ORM," Hibernate Project, 2026. [Online]. Available: <https://hibernate.org/orm/documentation/>
- [3] I. Roztropiński, "Optimistic vs Pessimistic Locking: concurrency control, conflicts, lost updates, retries and blocking," 2025.
- [4] "PostgreSQL 18 Documentation: 13.3. Explicit Locking," PostgreSQL Global Development Group, 2026. [Online]. Available: <https://www.postgresql.org/docs/>
- [5] "PostgreSQL 18 Documentation: Deadlocks," PostgreSQL Global Development Group, 2026. [Online]. Available: <https://www.postgresql.org/docs/>
- [6] "Spring Framework Documentation: Transaction Management," Broadcom Inc., 2026. [Online]. Available: <https://docs.spring.io/spring-framework/reference/>
- [7] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, "The notions of consistency and predicate locks in a database system," *Commun. ACM*, 1976.
- [8] D. Martinenghi, "Towards a visually interpretable analysis of Two-Phase Locking membership," *arXiv:2501.06978v1*, 2025.
- [9] W. H. Hesselink, P. Mathijssen, and M. Vervoort, "First-Come-First-Served as a Separate Principle," *ACM Trans. Parallel Comput.*, 2024.